第 6 章練習解答　CPU 排程

> 題號與本章練習題 PDF 一一對應。答案保留成立條件、關鍵推導與必要的反例。

OS06-E01　非搶占排程計算

1. FCFS：完成 8、12、14；等待 0、8、12；周轉 8、12、14；平均等待 20/3≈6.67。SJF 順序 P3,P2,P1：完成 2、6、14；等待 0、2、6；周轉 2、6、14；平均等待 8/3≈2.67。
2. SJF 平均等待低 4，時間單位依 burst 定義。FCFS 總周轉為 8+12+14=34，平均 34/3≈11.33；SJF 總周轉為 2+6+14=22，平均 22/3≈7.33，兩者並不相同。
3. 它要求所有工作的 burst time 已知，或至少順序選擇時的估計可反映實際需求；若估計錯誤，不能保證這個時間表。

檢查。 三個表格欄位都要對齊完成、等待與周轉。

OS06-E02　Round Robin

1. P1[0,2), P2[2,4), P3[4,6), P1[6,8), P3[8,9), P1[9,10)。完成時間 P1=10、P2=4、P3=9。
2. 等待時間等於總時間減 burst：P1=5、P2=2、P3=6；回應時間 P1=0、P2=2、P3=4。平均等待 13/3≈4.33。
3. q=1 會讓 P2/P3 更早得到首次回應，但行程更多，context switch 成本上升；完成順序與各工作局部執行片段可能改變。

檢查。 時間表每個區間不重疊，總長應等於所有 burst 加總。

OS06-E03　優先權與 starvation

1. 若新工作持續使低優先權者無法在有限次排程中前進，則 starvation 可發生；單一觀察到長等待不足以界定無界，需要 workload 模型與等待證明。
2. aging 把等待時間轉成有效 priority 增量。若等待每增加 W 時間，優先權至少上升固定量，便可能在有限界後追上。
3. 需要明確 aging 函數、tie-break 與最大等待上限，並含搶占與新工作 arrival 的最壞情況分析。

檢查。 平均成績好不代表 fairness 保證。

第 6 章練習解答　CPU 排程 （續）

OS06-E04　HRTICK CFS 逐事件推演

1. n=2≤8，所以 period=6 ms；等權重時 ideal_runtime=6×1024/2048=3 ms。nice=0 的 `calc_delta_fair` 不縮放，因此每執行 1 ms，vruntime 增加 1 ms。
2. 以下記錄事件處理與新current選定後的狀態，ready不含current，min_vruntime不倒退。t=0：current=P1，ready={P2@0}，min=0；t=3 HRTICK：current=P2，ready={P1@3}，min=0；t=6 HRTICK：current=P1，ready={P2@3}，min=3；t=9 wake/HRTICK：P3放回vruntime0，current=P3，ready={P2@3,P1@6}，min=3，原current領先6>1；t=11 P3完成：current=P2，ready={P1@6}，min=3；t=12 P2完成：current=P1，ready={}，min=6；t=14 P1完成：current無、ready={}，min保留最後更新值8。相同vruntime時沿題設入樹次序選取，不另加t=8切換。
3. Gantt 為 P1[0,3)、P2[3,6)、P1[6,9)、P3[9,11)、P2[11,12)、P1[12,14)。完成 P1/P2/P3=14/12/11。t=8 沒有 HRTICK、週期 tick、completion 或 wakeup，所以不切換。
4. P3 的門檻為 `max(舊 vruntime, min_vruntime−3)=max(0,3−3)=0`。喚醒比較在 t=9 做，P1 的 vruntime 已到 6；差值 6 ms 嚴格大於 1 ms，所以 P3 在 t=9 立即取得 CPU，不是在 t=8 預先切換。

檢查。 所有區間應由 slice 期滿、completion 或 wake 的嚴格條件產生；Gantt 不是獨立輸入。

OS06-E05　週期 tick 的嚴格大於

1. 從 t=0 起的 delta_exec 在 t=3 為 3、t=4 為 4。v3.2 的第一條件是 `delta_exec > ideal_runtime`；3>3 為 false，4>3 才成立。
2. P1 跑 [0,4)，P2 跑 [4,8) 並正好完成；P1 在 [8,9) 跑 1 ms。P3 在 t=9 放回 vruntime=2，與 P1 vruntime=5 相差 3>1，於是 [9,11) 搶占；P1 在 [11,14) 完成。completion=14/8/11。
3. ideal_runtime 是排程器允許的 slice 上限，不是硬體 timer 保證。只有 HRTICK 的 queued tick 會在精確 slice 邊界直接重排；一般 tick 只在解析度允許的檢查點用嚴格大於判斷。
4. t=8.5 發生 wake preemption，舊 CFS HRTICK 的 P1[6,9) 先改成 P1[6,8.5)，接著 P3[8.5,10.5)、P2[10.5,11.5)、P1[11.5,14)；全部 completion 改為 P1=14、P2=11.5、P3=10.5。

檢查。 嚴格大於、HRTICK queued path 與 wakeup comparison 是三個不同入口。

第 6 章練習解答　CPU 排程 （續）

OS06-E06　CPU 公平與同步語意分離

1. 總 weight=1359；A 的 slice=6×1024/1359≈4.521 ms，B 的 slice≈1.479 ms。CPU share 分別為約 75.35% 與 24.65%，不是固定相同 quantum。
2. 不能。若 A、B 各執行 ordinary read-modify-write `counter++`，同一個 CFS runqueue 仍可能讓兩者的 load/add/store 交錯，最後少於 2。Scheduler fairness 只決定 CPU 時間比例，不使複合資料操作原子。
3. B 必須在釋放 store 帶 release semantics，A 必須在觀察 ready 的 load 帶 acquire semantics；或使用具有等價 ordering 的 mutex/condition-variable 協定。只有 ready 的數值變 1，不能證明 buffer 可見。
4. mutex 提供同一 critical section 的 mutual exclusion；若 API 明確規定 lock acquire 與 unlock release 的 memory-ordering，也提供對應可見性。單說「有 mutex」仍要指定等待、釋放、錯誤路徑與 publication 是否在同一保護範圍。
5. memory barrier 只限制記憶體操作的排序或可見性，不建立兩 threads 進入區段的互斥；atomic operation 讓指定位置不可分割，並可依 release/acquire/seq_cst contract 發布另一位置的資料。mutex 以 ownership 保護整段 critical section，成功 lock/unlock 通常同時建立互斥與 happens-before。binary semaphore 初值 1 可讓一個 wait 取得 permit，因而序列化區段，但沒有 owner；counting semaphore 控制 permit 數量，是否提供 memory ordering 取決於 API。

檢查。 答案要把 scheduling share、atomicity、mutual exclusion 與 memory ordering 分成四個可獨立檢查的保證。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 6 章練習解答　CPU 排程 （續）

原創驗算程式附錄: scheduler_demo.py

```
"""Small deterministic CPU scheduling models.

Run:
    python scheduler_demo.py
"""

from dataclasses import dataclass


@dataclass(frozen=True)
class Job:
    name: str
    burst: int


def metrics(order: list[Job]) -> dict[str, float]:
    elapsed = 0
    waits: list[int] = []
    responses: list[int] = []
    turnarounds: list[int] = []
    first_start = {}
    for job in order:
        waits.append(elapsed)
        first_start[job.name] = elapsed
        elapsed += job.burst
        responses.append(first_start[job.name])
        turnarounds.append(elapsed)
    return {
        "avg_wait": sum(waits) / len(waits),
        "avg_response": sum(responses) / len(responses),
        "avg_turnaround": sum(turnarounds) / len(turnarounds),
    }


def round_robin(jobs: list[Job], quantum: int) -> dict[str, int]:
    remaining = {job.name: job.burst for job in jobs}
    elapsed = 0
    response: dict[str, int | None] = {job.name: None for job in jobs}
    completion: dict[str, int] = {}
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第 6 章練習解答　CPU 排程 （續）

```
    queue = list(jobs)
    while queue:
        job = queue.pop(0)
        if response[job.name] is None:
            response[job.name] = elapsed
        run = min(quantum, remaining[job.name])
        elapsed += run
        remaining[job.name] -= run
        if remaining[job.name] == 0:
            completion[job.name] = elapsed
        else:
            queue.append(job)
    return {
        "completion_P1": completion["P1"],
        "completion_P2": completion["P2"],
        "completion_P3": completion["P3"],
        "response_P3": int(response["P3"]),
    }


def main() -> None:
    jobs = [Job("P1", 8), Job("P2", 4), Job("P3", 2)]
    fcfs = metrics(jobs)
    sjf = metrics(sorted(jobs, key=lambda job: job.burst))
    rr = round_robin(jobs, quantum=2)
    assert round(fcfs["avg_wait"], 2) == 6.67
    assert round(sjf["avg_wait"], 2) == 2.67
    assert rr["completion_P1"] == 14
    assert rr["response_P3"] == 4
    print("FCFS", fcfs)
    print("SJF", sjf)
    print("RR", rr)


if __name__ == "__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study